

UNIVERSIDAD DEL DESARROLLO
FACULTAD DE INGENIERÍA

SEGUNDO INFORME CERTAMEN 2
ARQUITECTURA DE SISTEMAS
“MISIÓN EMPRENDE”

Profesor: Angel Rodrigo Nuñez Lopez

Equipo: Sebastián Ruiz, Leandro Añasco y Lucas Riquelme

Carrera: Ingeniería Civil Informática e Innovación Tecnológica

Fecha: 29/07/26

Índice

1. Resumen ejecutivo	3
2. Descripción general del sistema	3
2.1. Actores y responsabilidades	4
2.2. Fases principales de la experiencia	4
2.3. Flujo operativo de una sesión	4
3. Operatividad y cumplimiento funcional	5
3.1. Módulos desplegados	5
3.2. Stack tecnológico actual	6
3.3. Requisitos funcionales cubiertos	6
4. Arquitectura serverless y Clean Architecture	7
4.1. Regla de dependencia	7
4.2. Patrón de tres archivos por módulo	8
4.3. Estructura de directorios y archivos	8
4.4. Frontend como objeto de presentación	10
4.5. Inyección de dependencias y pruebas aisladas	10
4.6. Máquina de estados canónica	10
5. Infraestructura y despliegue en AWS	10
5.1. Servicios activos	10
5.2. Infraestructura como Código	11
5.3. Modelo single-table en DynamoDB	11
5.4. Entornos y pipeline	11
6. Patrones de diseño y resiliencia	12
6.1. Patrones implementados y JWT Authorizers	12

6.1.1. Outbox + De-duplication (Bandeja de salida y Desduplicación)	12
6.1.2. Throttling (Rate Limiting)	12
6.1.3. JWT Authorizers (Aislamiento por identidad)	12
6.2. Patrones planificados	12
6.2.1. Procesamiento asíncrono y Dead Letter Queue	12
6.2.2. Retry con backoff exponencial y jitter	13
6.2.3. Circuit Breaker	13
6.3. Patrones diferidos por escala	13
7. Estado de avance del proyecto	14
7.1. Resumen del trabajo hecho	14
7.2. Flujo ejecutable actual	14
7.3. Pendientes prioritarios	15
7.4. Decisiones que guían el trabajo pendiente	15
8. Conclusiones	15

1. Resumen ejecutivo

Misión Emprende UDD es una plataforma educativa gamificada que permite a profesores de la Universidad del Desarrollo conducir sesiones de emprendimiento con grupos de estudiantes. La experiencia organiza el trabajo en fases asociadas a colaboración, empatía, creatividad, comunicación y evaluación, utilizando tokens, temporizadores sincronizados, rankings y control centralizado del avance.

Desde el primer informe, el proyecto experimentó un cambio arquitectónico sustantivo. La solución dejó de estar centrada exclusivamente en el monolito Django y avanzó hacia un backend serverless desarrollado con Node.js y TypeScript, desplegado mediante AWS SAM. La arquitectura actual utiliza API Gateway, funciones Lambda independientes y DynamoDB, mientras el frontend permanece como una capa liviana de HTML, CSS y JavaScript que consume las respuestas del backend.

El avance demuestra la aplicación práctica de Clean Architecture en un entorno sin servidor. Los módulos se estructuran mediante adaptadores de entrada, servicios de dominio y repositorios de salida, manteniendo las reglas de negocio aisladas de HTTP y del SDK de AWS. El flujo operativo cubre acceso, creación de sesiones, control de fases, temporizadores, Sopa de Letras, Bubble Map y rankings parciales; los principales pendientes corresponden al almacenamiento real de fotografías LEGO, la fase de pitch y evaluación cruzada, y el endurecimiento productivo de la infraestructura (es decir la preparación para ser puesto en producción).

Aspecto	Estado al Certamen 2
Backend	Node.js 22.x y TypeScript compilado a JavaScript mediante esbuild.
Arquitectura	Serverless sobre AWS, organizada según Clean Architecture.
Persistencia	DynamoDB con diseño single-table y un índice secundario global.
Operatividad	Acceso, sesiones, transiciones, temporizadores, fases 1 y 2 operativas; fase 3 parcial.
Despliegue	Infraestructura como Código mediante AWS SAM.

2. Descripción general del sistema

Misión Emprende mantiene el propósito pedagógico descrito en el primer informe: facilitar el aprendizaje de emprendimiento e innovación mediante una experiencia grupal, narrativa y gamificada. El profesor crea y controla la sesión, mientras los estudiantes participan mediante códigos de acceso asociados a cada grupo. La plataforma coordina el avance, aplica las reglas de asignación de tokens y consolida los resultados de cada fase.

2.1. Actores y responsabilidades

Rol	Responsabilidad
Administrador	Configura el catálogo de temáticas y desafíos, y mantiene los contenidos generales del juego.
Profesor	Crea sesiones, forma grupos, controla las transiciones, supervisa los temporizadores y conduce la evaluación.
Grupo de estudiantes	Ingresa mediante un código, desarrolla las actividades, acumula tokens y consulta su progreso.

2.2. Fases principales de la experiencia

Fase	Actividad	Propósito
Fase 1	Sopa de Letras	Fortalecer el trabajo en equipo mediante la búsqueda de conceptos clave y la asignación idempotente de tokens.
Fase 2	Bubble Map de Empatía	Construir colaborativamente la comprensión del usuario y del problema seleccionado.
Fase 3	Creatividad LEGO	Elaborar un prototipo físico y registrar evidencia fotográfica de la solución.
Fase 4,5 y 6	Pitch y evaluación cruzada	Presentar la propuesta, evaluar a otros grupos y consolidar el ranking final.

2.3. Flujo operativo de una sesión

El profesor se autentica y crea una sesión con grupos y códigos únicos. Cada grupo ingresa utilizando su código y recibe un token JWT que identifica la sesión y el equipo. Desde ese momento, los dispositivos consultan periódicamente el estado de la sesión, incluyendo la fase actual, el término del temporizador y los tokens acumulados.

Las transiciones son controladas exclusivamente por el profesor. Cuando se solicita avanzar, el backend valida que el estado de destino corresponda al sucesor permitido por la máquina de estados. Durante las actividades, las acciones de los grupos se procesan en funciones Lambda específicas y se persisten en DynamoDB. Si un grupo intenta acceder a una fase distinta de la activa, el sistema responde con un conflicto y evita la ejecución de reglas fuera de contexto.

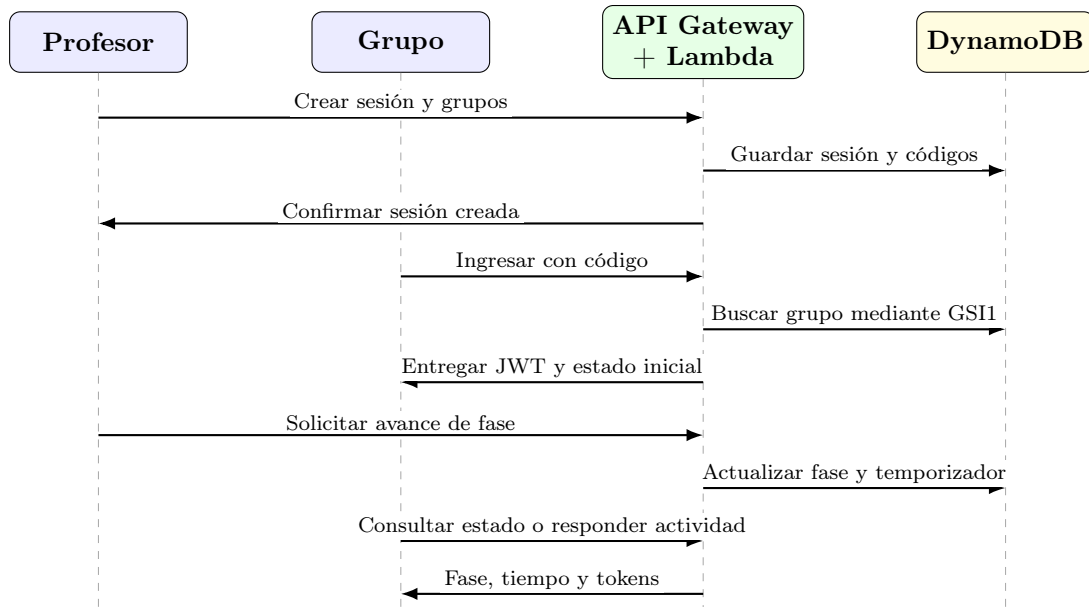


Figura 1: Flujo simplificado de una sesión operativa

3. Operatividad y cumplimiento funcional

3.1. Módulos desplegados

La solución se divide en funciones asociadas a capacidades concretas. Esta separación reduce el alcance de cada despliegue y permite evolucionar una fase sin convertir el backend en un controlador central de gran tamaño.

Módulo	Endpoint principal	Estado
Acceso	POST /api/acceso/ingresar	Operativo.
Administrador	POST /api/admin/ingresar	En revisión.
Profesor	POST /api/profesor/sesiones y /acciones	Operativo.
Sesiones	GET /api/sesiones/actual	Operativo.
Fase 1	POST /api/fase1/palabras; GET /ranking	Operativo.
Fase 2	POST /api/fase2/bubblemap; completar; ranking	Operativo.
Fase 3	POST /api/fase3/confirmar-foto	Parcial, sin almacenamiento S3 real.
Fase 4, 5 y 6	Pitch y evaluación cruzada	Planificado.
Catálogo	Temáticas y desafíos adminis-trables	Planificado.

3.2. Stack tecnológico actual

Componente	Tecnología	Observación
Runtime	Node.js 22.x	Runtime de las funciones Lambda.
Lenguaje	TypeScript 7.0.2	Se utiliza para verificación estática; es compilado a JavaScript.
Empaquetado	esbuild 0.28.1	Genera artefactos compactos para Lambda.
Infraestructura como Código	AWS SAM	Define funciones, API y persistencia.
Persistencia	DynamoDB y AWS SDK v3	Modelo single-table con acceso encapsulado.
Pruebas	Vitest 4.1.10	Ejecuta pruebas unitarias sin infraestructura AWS.
Frontend	HTML, CSS y JavaScript	Capa de presentación sin reglas de negocio.

3.3. Requisitos funcionales cubiertos

Los requisitos más relevantes ya operativos corresponden al acceso mediante código y JWT, el rate limiting por dirección IP, el aislamiento de sesiones por profesor, la creación de grupos con códigos únicos, las transiciones controladas, la máquina de estados centralizada, los temporizadores sincronizados y las mecánicas de las fases 1 y 2.

Requisito	Capacidad	Estado
FR-003–005	Acceso seguro, rate limiting y aislamiento por profesor.	Implementado.
FR-011	Creación de sesiones, grupos y códigos únicos.	Implementado.
FR-015–017	Transiciones exclusivas del profesor y validación de fase.	Implementado.
FR-019–023	Temporizadores sincronizados mediante tiempo del servidor.	Implementado.
FR-024–028	Sopa de Letras, tokens y ranking de fase 1.	Implementado.
FR-029–032	Bubble Map, recompensa y ranking de fase 2.	Implementado.
FR-033–038	Flujo LEGO, fotografía y tokens de fase 3.	Parcial.
FR-052–053	Consulta periódica del estado de sesión.	Implementado.

4. Arquitectura serverless y Clean Architecture

4.1. Regla de dependencia

La arquitectura aplica como principio invariante que las dependencias deben apuntar hacia el núcleo. Las reglas de tokens, las transiciones, las validaciones de fase y los rankings no conocen API Gateway, eventos Lambda, DynamoDB ni detalles HTTP. Los mecanismos externos se conectan mediante adaptadores que traducen entradas y salidas sin desplazar decisiones de negocio hacia la infraestructura.

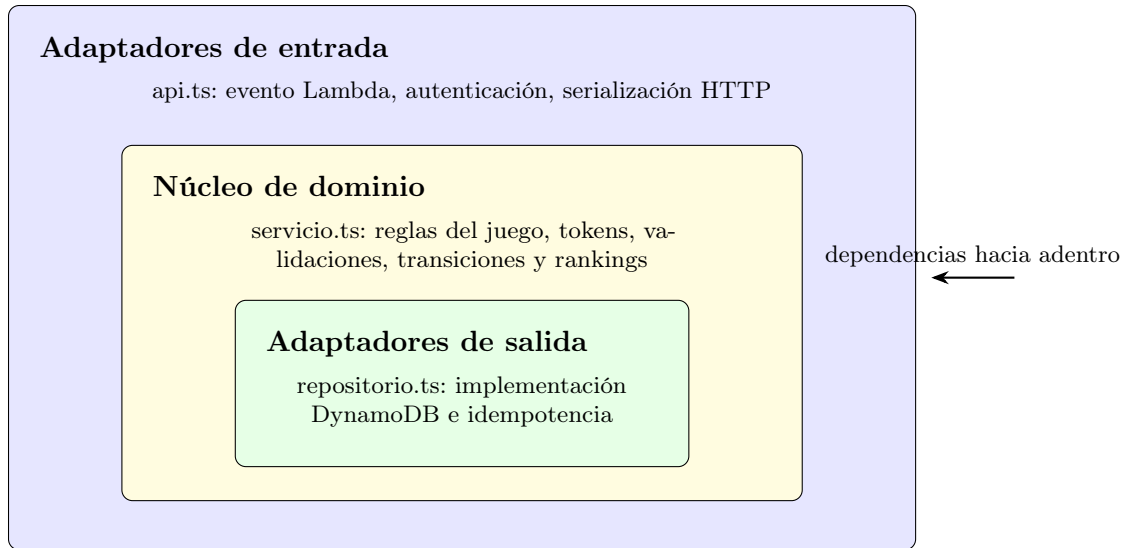


Figura 2: Aplicación de Clean Architecture en el backend serverless

4.2. Patrón de tres archivos por módulo

Cada módulo funcional mantiene tres responsabilidades explícitas:

- **api.ts**: actúa como adaptador de entrada, interpreta el evento, extrae el contexto de seguridad, invoca el servicio y construye la respuesta HTTP.
- **servicio.ts**: implementa los casos de uso y las reglas del juego sin importar el SDK de AWS ni conocer la forma del evento Lambda.
- **repositorio.ts**: concentra el acceso a DynamoDB, los prefijos de claves, las expresiones condicionales y la implementación concreta de las interfaces requeridas por el servicio.

Los componentes transversales residen en `compartido/`, donde se centralizan la conexión a datos, las respuestas, la seguridad y la máquina de estados. Como regla de diseño, ningún módulo funcional importa directamente el repositorio de otro. Si una fase necesita resultados de una fase anterior, debe hacerlo mediante un contrato explícito compartido.

4.3. Estructura de directorios y archivos

La organización física del proyecto refleja la separación de responsabilidades descrita anteriormente. El backend serverless agrupa cada capacidad funcional en un módulo independiente y reserva el directorio `compartido/` para contratos y utilidades transversales. La estructura incluye la definición de infraestructura, la configuración de TypeScript, los módulos funcionales y las pruebas automatizadas.

La siguiente representación, presentada como una salida de terminal, ejemplifica la estructura relevante del repositorio, a nivel de backend:

```

PROYECTO-ARQUITECTURA/
|-- backend-serverless/
|   |-- template.yaml
|   |-- package.json
|   |-- tsconfig.json
|   |-- src/
|       |-- compartido/
|           |-- baseDatos.ts
|           |-- respuestas.ts
|           |-- seguridad.ts
|           |-- maquinaEstados.ts
|           |-- contratos/
|               |-- acceso/
|                   |-- api.ts
|                   |-- servicio.ts
|                   '-- repositorio.ts
|               |-- profesor/
|                   |-- api.ts
|                   |-- servicio.ts
|                   '-- repositorio.ts
|               |-- sesiones/
|                   |-- api.ts
|                   |-- servicio.ts
|                   '-- repositorio.ts
|               |-- fase1/
|                   |-- api.ts
|                   |-- servicio.ts
|                   '-- repositorio.ts
|               |-- fase2/
|                   |-- api.ts
|                   |-- servicio.ts
|                   '-- repositorio.ts
|               '-- fase3/
|                   |-- api.ts
|                   |-- servicio.ts
|                   '-- repositorio.ts
|   '-- pruebas/
|       |-- acceso.test.ts
|       |-- profesor.test.ts
|       |-- fase1.test.ts
|       '-- fase2.test.ts

```

En esta estructura, los archivos `api.ts` representan los adaptadores de entrada; `servicio.ts` contiene los casos de uso y reglas del juego; y `repositorio.ts` implementa la comunicación con DynamoDB. Las carpetas futuras, como `fase4-5-6/` y `catalogo/`, deberán incorporarse siguiendo el mismo patrón, sin importar repositorios de otros módulos y utilizando contratos explícitos cuando sea necesario compartir información.

4.4. Frontend como objeto de presentación

El frontend utiliza HTML, CSS y JavaScript sin framework. Su función es conservar el token, invocar endpoints y renderizar respuestas. La validación de palabras, la asignación de tokens, la idempotencia, el cálculo de rankings, las transiciones y la propiedad de las sesiones permanecen en el backend.

El contador visible constituye una excepción deliberada: el navegador calcula los segundos restantes a partir de `timerFin`, pero dicho valor proviene del servidor. De esta forma, la presentación puede actualizarse localmente sin convertir al dispositivo en fuente de verdad.

4.5. Inyección de dependencias y pruebas aisladas

La naturaleza efímera de Lambda hace innecesario un contenedor IoC persistente. La implementación concreta del repositorio se crea en el handler y se entrega como parámetro al servicio. El servicio depende de una interfaz, no de la clase DynamoDB. Esta inyección explícita permite reemplazar el repositorio real por un objeto TypeScript simple durante las pruebas.

Las pruebas de negocio verifican, entre otros comportamientos, que una palabra nueva entregue un token y que una segunda llamada con la misma palabra no modifique el puntaje. Debido a que el núcleo no importa DynamoDB ni AWS, estas pruebas se ejecutan localmente con Vitest y sin librerías de mocking de infraestructura.

4.6. Máquina de estados canónica

La secuencia completa del juego reside en `compartido/maquinaEstados.ts`. Los módulos no utilizan strings independientes para representar fases, lo que evita divergencias entre rutas y reglas. Solo el profesor puede disparar transiciones y el backend valida que el destino sea el sucesor directo del estado actual. Los saltos ilegales son rechazados antes de alterar la sesión.

5. Infraestructura y despliegue en AWS

5.1. Servicios activos

La solución opera sobre servicios administrados de AWS. CloudFront y el bucket del frontend se encuentran contemplados en la arquitectura de producción; API Gateway distribuye las solicitudes a funciones Lambda especializadas y DynamoDB mantiene el estado del juego. El almacenamiento S3 para fotografías LEGO y la cola de procesamiento todavía forman parte del trabajo pendiente.

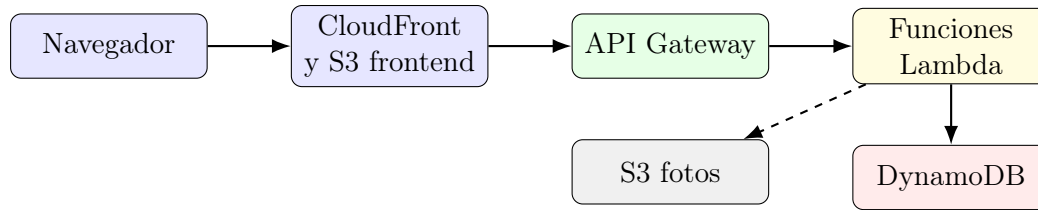


Figura 3: Infraestructura serverless actual y componentes planificados

5.2. Infraestructura como Código

La infraestructura se define en `backend-serverless/template.yaml` mediante AWS SAM. El archivo declara la tabla DynamoDB, el índice GSI1, las funciones, los permisos, las rutas HTTP, la memoria y los tiempos máximos de ejecución. La consola se utiliza para observación, no como mecanismo principal de creación de recursos.

Esta práctica permite reproducir ambientes, revisar los cambios de infraestructura junto con el código y disminuir las diferencias entre desarrollo y producción. La tabla funciona con capacidad *On-Demand*, cifrado en reposo y nombres parametrizados por entorno.

5.3. Modelo single-table en DynamoDB

El sistema utiliza una tabla por entorno. Sesiones, grupos y alumnos se distinguen mediante prefijos de clave que solo conocen los repositorios.

Entidad	Clave primaria	Acceso secundario
Sesión	PK = SESSION#id; SK = METADATOS	GSI1 por PROFE-SOR#profesorId.
Grupo	PK = SESSION#id; SK = GRUPO#grupoId	GSI1 por CODIGO#codigoAcceso.
Alumno	PK = SESSION#id; SK = ALUMNO#alumnoId	Asociación al grupo dentro de la sesión.

El índice GSI1 resuelve dos búsquedas críticas: listar las sesiones pertenecientes a un profesor y encontrar un grupo a partir de su código de acceso. Las reglas de prefijos y consultas permanecen en `repositorio.ts`, evitando que los servicios conozcan la estructura física de almacenamiento.

5.4. Entornos y pipeline

Los ambientes `dev` y `prod` se separan mediante parámetros SAM y tablas independientes. En producción se proyecta habilitar recuperación a un punto en el tiempo y restringir CORS al dominio servido por CloudFront.

El pipeline definido considera la verificación de tipos, las pruebas Vitest, el empaquetado con esbuild,

la validación SAM y el despliegue progresivo. Debido a las credenciales rotativas y restricciones de AWS Academy, actualmente los pasos se ejecutan manualmente. La secuencia es compatible con una automatización futura mediante GitHub Actions cuando exista un mecanismo de autenticación estable.

6. Patrones de diseño y resiliencia

El proyecto distingue entre patrones implementados, parcialmente implementados y planificados. Esta separación permite evaluar la arquitectura según evidencia real y evita presentar como finalizadas capacidades que aún dependen de historias pendientes.

6.1. Patrones implementados y JWT Authorizers

6.1.1. Outbox + De-duplication (Bandeja de salida y Desduplicación)

Las operaciones que entregan tokens utilizan expresiones condicionales de DynamoDB. Si una palabra o recompensa ya fue registrada, la base de datos rechaza la escritura duplicada y el puntaje permanece intacto. La idempotencia protege el resultado ante dobles clics, reintentos de red y reconexiones, condiciones frecuentes en una sesión con múltiples dispositivos.

6.1.2. Throttling (Rate Limiting)

El acceso registra intentos fallidos por dirección IP con un TTL de cinco minutos y bloquea temporalmente después de cinco intentos. Esta medida reduce el riesgo de fuerza bruta sobre códigos alfanuméricos de seis caracteres. Se mantiene documentada una deuda menor: los códigos vacíos todavía no consumen el contador porque la validación de formato ocurre antes del incremento.

6.1.3. JWT Authorizers (Aislamiento por identidad)

Las rutas de profesor obtienen `profesorId` exclusivamente desde el token y lo utilizan como condición obligatoria en GSI1. El identificador no se acepta desde el cuerpo ni desde parámetros manipulables. Esta decisión evita que un profesor autenticado consulte o modifique sesiones ajenas.

6.2. Patrones planificados

6.2.1. Procesamiento asíncrono y Dead Letter Queue

El flujo proyectado para las fotografías LEGO utiliza S3, SQS y una Lambda de procesamiento. La confirmación del grupo genera una respuesta inmediata, mientras la imagen se valida y transforma

de forma asíncrona. Después de tres fallos, el mensaje se deriva a una DLQ para conservar evidencia sin bloquear el avance del juego.

6.2.2. Retry con backoff exponencial y jitter

El SDK ya incorpora reintentos por defecto, pero se planifica formalizar tres intentos y una espera exponencial con variación aleatoria. Esta configuración busca reducir reintentos sincronizados cuando existen picos o fallos transitorios de DynamoDB.

6.2.3. Circuit Breaker

El patrón se proyecta para integraciones como la confirmación de fotografías y la obtención de secretos. Su implementación depende de observabilidad y métricas confiables, puesto que los umbrales de apertura y recuperación deben responder a evidencia operacional y no a valores arbitrarios.

6.3. Patrones diferidos por escala

Patrón	Estado	Justificación
Cache-Aside	No planificado	DynamoDB On-Demand absorbe la carga actual; ElastiCache agregaría costo y complejidad injustificados para aproximadamente 10 grupos por 3 sesiones.
Event Sourcing	Diferido	La reconstrucción de estado y los snapshots exceden la necesidad de trazabilidad actual.
CQRS	No planificado	El volumen aproximado de 10 solicitudes por segundo no justifica separar modelos de lectura y escritura.
EventBridge para rankings	Diferido	El cálculo síncrono sobre pocos grupos tiene latencia despreciable y evita infraestructura adicional.

7. Estado de avance del proyecto

7.1. Resumen del trabajo hecho

Al 29 de julio de 2026 el backend se encuentra desarrollado en un 80 % y el frontend en un 50 %. Este estado refleja que el núcleo operativo está desplegado, pero la sesión completa todavía depende de la finalización de LEGO, Pitch, catálogo y capacidades productivas. Para una mejor comprensión se desglosa el avance en épicas y número de historias completadas por cada una.

Épica	Avance	Estado principal
1. Identidad y acceso seguro	4 de 5 terminadas	Autenticación de administrador en revisión.
2. Orquestación de sesión	2 de 5 terminadas	Ajuste de grupos, cierre y salud en progreso.
3. Catálogo de temáticas	0 de 3 iniciadas	Backlog; contenido aún definido en frontend.
4. Fases 1 y 2	3 de 4 terminadas	Falta endpoint dedicado para progreso del profesor.
5. Fase 3 LEGO	0 de 3 terminadas	Confirmación parcial; faltan S3.
6. Pitch y evaluación	No iniciada	Backlog; falta módulo y ranking final.
7. Preparación productiva	0 de 6 terminadas	Infraestructura, seguridad y observabilidad en desarrollo.

7.2. Flujo ejecutable actual

Actualmente un profesor puede autenticarse, crear una sesión con grupos y códigos, permitir el ingreso de los estudiantes, avanzar por la máquina de estados, iniciar temporizadores sincronizados, ejecutar la Sopa de Letras, consultar el ranking de fase 1, desarrollar el Bubble Map y consultar el estado global mediante polling. Las recompensas de las fases implementadas son persistentes e idempotentes.

La fase LEGO posee una implementación parcial: el frontend conserva temporalmente la imagen y el backend acepta la confirmación para entregar tokens, pero todavía no verifica un objeto real en S3. Esta diferencia entre demostración funcional e integración productiva se encuentra documentada como deuda técnica y constituye una prioridad alta.

7.3. Pendientes prioritarios

Prioridad	Pendiente	Impacto
Alta	Flujo S3 real para fotografías LEGO.	Completar evidencia y persistencia de la fase 3.
Alta	Módulo de Pitch y evaluación cruzada.	Permitir finalizar la experiencia pedagógica.
Alta	Ranking acumulativo y estado finalizado.	Cerrar formalmente la sesión y consolidar resultados.
Media	Catálogo administrable.	Eliminar temáticas hardcodeadas en el frontend.
Media	CloudFront, bucket frontend, PITR y CORS productivo.	Completar el despliegue endurecido.
Baja	Logs estructurados, alarmas y pruebas de carga.	Validar observabilidad y atributos no funcionales.

7.4. Decisiones que guían el trabajo pendiente

Las nuevas capacidades deben respetar las decisiones arquitectónicas ya adoptadas: evitar imports cruzados entre repositorios, centralizar los estados, extraer la identidad desde los tokens, mantener rankings síncronos mientras la escala lo permita, orientar las dependencias hacia el núcleo y conservar una única tabla DynamoDB con GSI. Asimismo, ningún nuevo handler (entiendase, componente de software encargado de recibir una solicitud) debe interpretar tokens manualmente fuera de los helpers de seguridad compartidos.

Estas decisiones reducen la posibilidad de que las historias pendientes reintroduzcan el acoplamiento observado en la versión monolítica. La fase 4, 5 y 6 y el catálogo deberán seguir el mismo patrón `api.ts-servicio.ts-repositorio.ts`, incorporando contratos compartidos cuando necesiten datos de otros módulos.

8. Conclusiones

El segundo avance de Misión Emprende demuestra una evolución arquitectónica verificable. El sistema ya no se limita a una reorganización conceptual, sino que opera con un backend serverless desplegado, módulos separados, persistencia DynamoDB y reglas de negocio testeables sin infraestructura. La adopción de Node.js, TypeScript, AWS SAM y funciones Lambda responde al objetivo de escalar sesiones sin administrar servidores y de aislar las reglas del juego respecto de los mecanismos externos.

La aplicación de Clean Architecture se observa en la dirección de dependencias, la división de responsabilidades y la inyección explícita de repositorios. Los patrones de idempotencia, rate limiting y aislamiento por identidad ya protegen operaciones críticas. Paralelamente, el proyecto documenta con transparencia los componentes incompletos y evita confundir una planificación arquitectónica con una implementación finalizada.

El principal desafío para el siguiente ciclo consiste en completar el recorrido pedagógico de extremo a extremo. Esto requiere implementar el flujo S3 y asíncrono de LEGO, desarrollar la fase de pitch y evaluación, consolidar el ranking final y cerrar la preparación productiva. La arquitectura existente entrega una base coherente para abordar estas historias sin regresar a controladores centrales ni acoplar el dominio directamente a AWS.